

FPGA I2C Driver for MCP4725 DAC

Project Technical Documentation

Version 1.0 | March 2026

1. Project Overview

This project implements a synthesizable Verilog I2C master controller on an FPGA to drive the Microchip MCP4725 12-bit Digital-to-Analog Converter. The design generates a standard 100 kHz I2C bus, formats a 4-byte write transaction according to the MCP4725 datasheet protocol, and shifts a 12-bit DAC code out over the bus.

The core design file (dac.v) contains three concurrent always blocks:

- An SCL clock generator that divides the system clock to produce the I2C clock
- An I2C FSM (Finite State Machine) that manages byte-by-byte transmission
- An SDA monitoring block that handles tri-stating during ACK windows

2. Hardware Setup

2.1 MCP4725 Pin Description

Pin	Name	Function
1	VOUT	Analog output (0 V to VDD, rail-to-rail)
2	VSS	Ground reference
3	VDD	Supply voltage (2.7 V – 5.5 V)
4	SDA	I2C serial data (open-drain, requires pull-up resistor)
5	SCL	I2C serial clock (open-drain, requires pull-up resistor)
6	A0	Address bit 0 — tie to VSS (logic 0) or VDD (logic 1)

2.2 I2C Address

The MCP4725 device code is fixed at 1100b. The full 7-bit address is:

$$\text{Address} = 1100 \text{ A2 A1 A0}$$

A2 and A1 are hard-wired at manufacturing (default: 00). A0 is set by the logic level of the A0 pin. With A0 tied to VSS, the address resolves to 0x60 (decimal 96), which matches the I2C_ADDR parameter in the Verilog source.

2.3 Pull-up Resistors

Both SDA and SCL are open-drain lines and require external pull-up resistors to VDD. A value of 4.7 k Ω is recommended for standard 100 kHz operation. The SCL and SDA pins of the FPGA must be connected to the MCP4725 through these resistors, with both pull-ups tied to the same VDD rail powering the DAC.

3. Module Interface

Port	Direction	Width	Description
I_clk	Input	1 bit	System clock (e.g. 25 MHz or 50 MHz)
I_rst_n	Input	1 bit	Active-low reset — triggers a new I2C transaction on assertion
DAC_VALUE	Input	12 bits	DAC code to transmit (0x000 = 0 V, 0xFFF = VDD)
O_scl	Output	1 bit	I2C SCL clock line driven to the MCP4725
IO_sda	Inout	1 bit	I2C SDA data line — bidirectional, open-drain

Asserting I_rst_n (pulling it low) initiates a complete 4-byte I2C write cycle. Releasing it (high) leaves the FSM in the DONE state holding SDA and SCL idle.

4. How the Code Works

4.1 SCL Clock Generator

The first always block generates the I2C SCL signal by dividing the system clock. A counter (scl_cnt) increments every clock cycle and toggles scl_reg when it reaches DIV-1, producing a symmetric 50% duty-cycle clock. SCL is only active when scl_en is asserted by the FSM.

All FSM state transitions are gated on `scl_cnt == 0` to ensure SDA changes only occur on SCL falling edges, satisfying I2C setup and hold time requirements.

$$\text{DIV} = \text{F_clk} / (2 \times \text{F_scl})$$

System Clock	Target SCL	Required DIV
25 MHz	100 kHz (Standard Mode)	125
50 MHz	100 kHz (Standard Mode)	250 (current default)
100 MHz	100 kHz (Standard Mode)	500 (widen counter to 10-bit)
50 MHz	400 kHz (Fast Mode)	62

4.2 I2C FSM

The second always block is the heart of the design. It steps through seven states on each reset assertion to complete one full I2C write transaction:

State	ID	Description
IDLE	0	Disables SCL, clears bit and byte counters, holds SDA high
START	1	Enables SCL; pulls SDA low while SCL is high to generate the I2C START condition
LOAD BYTE	2	Selects the correct byte for the current <code>byte_cnt</code> value and loads it into <code>byte_data</code>
SEND BYTE	3	Shifts out 8 bits MSB-first — SDA is updated on each SCL falling edge; <code>bit_cnt</code> increments on rising edge
ACK	4	Provides the 9th clock pulse for the slave ACK; increments <code>byte_cnt</code> ; loops back to LOAD BYTE or advances to STOP
STOP	5	Pulls SDA low then releases it high while SCL is high to generate the I2C STOP condition
DONE	6	Holds SDA high after the transaction is complete

4.3 Byte Transmission Sequence

The MCP4725 Write DAC Register command (`C2=0`, `C1=1`, `C0=0`) requires exactly 4 bytes per transaction. The FSM transmits them in order:

byte_cnt	Byte	Value / Expression	Purpose
0	Address + Write	0xC0 (0x60 shifted left, R/W=0)	Device addressing, write direction
1	Control Byte	0x40 (<code>C1=1</code> ,	Write DAC register, normal power mode

byte_cnt	Byte	Value / Expression	Purpose
		PD1=PD0=0)	
2	MSB Data	DAC_VALUE[11:4]	Upper 8 bits of the 12-bit DAC code
3	LSB Data	{DAC_VALUE[3:0], 4'b0000}	Lower 4 bits, zero-padded to a full byte

Once the 4th ACK pulse is received, the MCP4725 updates its analog output immediately. The output voltage is given by:

$$V_{OUT} = V_{DD} \times DAC_VALUE / 4096$$

4.4 SDA ACK Monitoring Block

The third always block tracks SCL edges using a one-cycle delayed copy of the SCL output (scl_delayed). A counter (count) increments on each SCL edge after the first edge is detected. This allows the block to identify the precise position within the I2C bit stream.

The signal high_imp controls the tri-state direction of IO_sda:

- high_imp = 1 — FPGA actively drives IO_sda (during all data bits)
- high_imp = 0 — IO_sda is released to high-impedance, allowing the MCP4725 to pull SDA low for ACK

SDA is released (high_imp = 0) at edge counts 16 and 17, which correspond to the 9th SCL clock pulse of each byte. The current implementation does not inspect the ACK value — error recovery on NACK is a planned future enhancement.

5. Configuration Parameters

Parameter	Default	Description
I2C_ADDR	7'd96 (0x60)	7-bit I2C address — update to match your MCP4725 A0/A1/A2 wiring
DIV	8'd250	SCL half-period divider — calculate as F_clk / (2 x F_scl)

6. Known Limitations

- No NACK detection — the ACK clock pulse is provided but the SDA value during the ACK window is not checked; the FSM proceeds regardless

- Single-shot operation — a reset pulse is required to initiate each new transmission; the module does not auto-retrigger on DAC_VALUE changes
 - 8-bit DIV counter — for system clocks above 25 MHz targeting 100 kHz SCL, DIV exceeds 255 and the scl_cnt register width must be increased
 - No multi-master support — the design assumes sole ownership of the I2C bus with no bus arbitration logic
 - No EEPROM write — only the fast DAC register write command is implemented; persistent storage to EEPROM is not yet supported
 - No Fast Mode support — the design targets 100 kHz Standard Mode only; 400 kHz Fast Mode requires DIV adjustment and timing margin verification
-

7. References

- Microchip Technology — MCP4725 Datasheet DS22039C
- NXP Semiconductors — I2C Bus Specification and User Manual UM10204
- IEEE Std 1364-2001 — Verilog Hardware Description Language